

INTERNSHIP REPORT

semanticClimate
Aleena Harold Peter
4th Year, 7th Semester · Information Science and Engineering
April – July 2026

Abstract

This report documents a twelve-week internship with semanticClimate (April–July 2026), covering evaluation of the Climate Academy chatbot, [frontend development](#) for its production version, and the creation of [ClimateInsight](#), a RAG chatbot based on the IPCC AR6 Synthesis Report.

Acknowledgement

I would like to express my sincere gratitude to **Dr. Peter Murray-Rust** for his invaluable guidance, patience, kindness, and constant encouragement throughout my internship. His willingness to answer my questions, clarify my doubts, and share his knowledge made this a truly rewarding learning experience.

I am grateful to **Dr. Gitanjali Yadav** for her guidance and supervision during the internship. I also thank **Dr. Renu Kumari** for her support and for patiently clarifying internship-related queries whenever required.

My sincere thanks go to **Ms. Udit Agarwal** for her valuable suggestions and technical guidance throughout the project. I also thank my fellow interns, **Mr. Saarthak Patwal** and **Ms. Shreya S. Bhat**, for their support and collaboration.

Finally, I extend my heartfelt thanks to the entire **semanticClimate** team for providing me with the opportunity to work on meaningful projects and gain invaluable experience in AI and climate technology.

1. Introduction and Context

This independent internship at semanticClimate focused on improving the organisation's climate chatbot ecosystem. The work included evaluating the existing Climate Academy chatbot, enhancing its production frontend, and developing **ClimateInsight**, a RAG chatbot grounded in the IPCC AR6 Synthesis Report under the supervision of Dr. Peter Murray-Rust and Dr. Gitanjali Yadav.

2. Phase One: Testing the Streamlit Chatbot

(Late April to mid-May | April 23 – May 15)

The internship began with environment setup and testing of the Streamlit-based chatbot. The first major challenge was a Python version incompatibility: the project required Python 3.11 or 3.12, while Python 3.14 caused failures across Streamlit and other dependencies. Diagnosing this issue established an important lesson: always verify core requirements before attempting higher-level fixes. Once the environment was stable, testing showed the chatbot handled climate-related queries well and rejected out-of-domain questions without

hallucination, while also revealing issues such as inconsistent responses, missing book images, and structural debt in the codebase.

Weeks 3 and 4 involved branch management, smoke testing the FastAPI backend, and setting up Cloudflare tunnels for collaborative remote testing. [Documented the setup of four concurrent terminal processes on Windows](#) for future contributors. A critical image rendering bug, such as images overflowing their containers in the student book viewer, was traced through the rendering pipeline and [fixed with responsive CSS](#) across FastAPI rendering modules.

By the end of Phase One, a thorough understanding had been built of the project's three-part modular architecture, the frontend-backend communication model, ChromaDB's role as a one-time-indexed vector store, and the specific failure modes users were encountering. This foundation was essential for the frontend work that followed.

3. Phase Two: Frontend Development of the Cloudflare Deployment and Refactoring Code

Mid-May to late-June (May 16 – June 25)

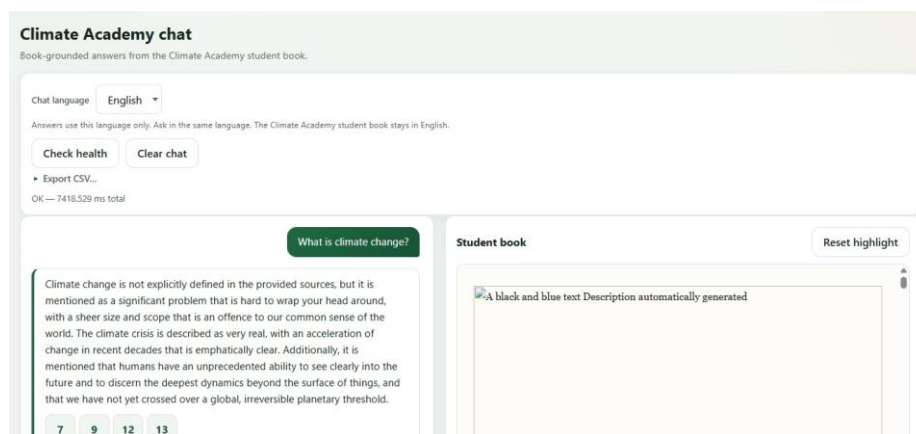


Figure 3.1. Base Prototype of the Cloudflare CABot

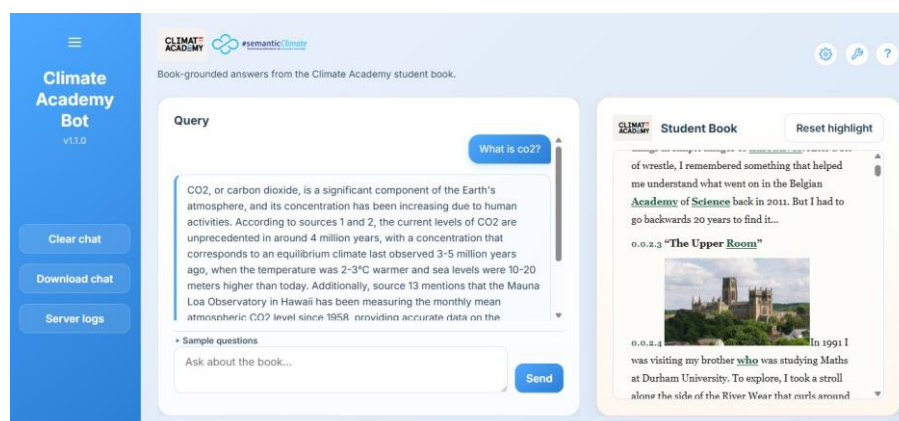


Figure 3.2. Redesigned Prototype of the Cloudflare CABot

In Week 5, the team had suggested a complete redesign of the Cloudflare-hosted frontend. The existing interface was functional but minimal, with a plain white layout, scattered settings, and little guidance for new users. Before implementation, the interface was carefully analyzed and a design concept prepared. This planning proved valuable when the initial [Help window implementation](#) caused regressions in other components and had to be rebuilt as an [isolated, self-contained module](#).

The major deliverable was [Version 1.1.1](#), implemented across Weeks 6 and 7. Key engineering decisions included: a two-column layout with a [collapsible sidebar](#) featuring a hamburger toggle and smooth CSS transitions; a design token system based on the Climate Academy colour palette, an animated thinking indicator, improved citation chips, reorganized settings and developer tools, and a modular CSS structure replaced the monolithic stylesheet. Accessibility was enhanced through keyboard navigation and consistent ARIA labelling.

Internationalisation received sustained attention throughout. The [localization file \(ui_strings.js\)](#) was extended across all five supported languages - English, French, Spanish, Portuguese, and Hindi. The Help window, once rebuilt, loaded documentation dynamically from a Markdown file via a lightweight Markdown-to-HTML renderer, so content could be updated without touching JavaScript.

Starting from mid-June through late June, the chatbot codebase was refactored into a modular, layered architecture with separate backend, frontend, data, documentation, and testing components. This improved the project's organization and maintainability while preserving its existing functionality.

By the end of Phase Two, the Cloudflare-hosted chatbot featured an improved interface, a maintainable CSS architecture, documented component patterns, and a refactored codebase with clearer separation of concerns, providing a stronger foundation for future development.

4. Phase Three: Building [ClimateInsight](#) - the IPCC AR6 Chatbot

Late-June to Present (June 26 – Present)

4.1 First Attempt and Lessons Learned



Figure 4.1. Climate Academy Chatbot (CABot), which inspired the development of ClimateInsight

The third phase was the independent design and construction of ClimateInsight - a RAG chatbot grounded in the IPCC AR6 Synthesis Report, inspired by the [CABot](#).

An initial prototype used Flask, Celery, Redis, ChromaDB, Ollama, and MiniLM as separate services. Although the RAG pipeline worked, integration issues and the complexity of managing multiple processes led to the approach being abandoned in favour of a simpler, more maintainable architecture.

4.2 The Rebuilt Architecture

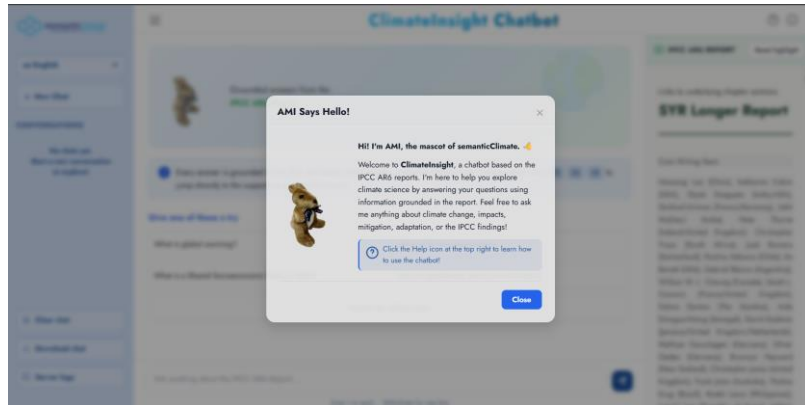


Figure 4.2.1. ClimateInsight Welcome Screen Featuring AMI, the semanticClimate Mascot.

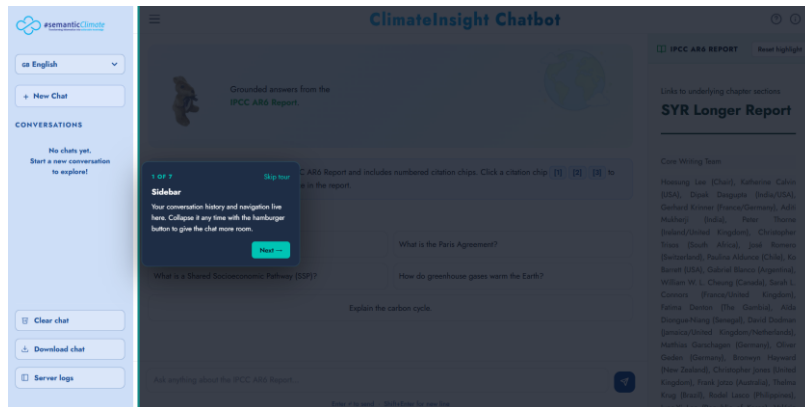


Figure 4.2.2. Interactive Help Tour of the ClimateInsight Interface



Figure 4.2.3. ClimateInsight Responding to a User Query (bot answering a question)

The second attempt rewrote the system from scratch with the single-terminal constraint as the primary design driver. Celery and Redis were dropped. The RAG pipeline ran synchronously within Flask. The frontend was plain HTML, CSS, and JavaScript with no build tooling. Each layer was verified before the next was added: Flask first, then MiniLM, then ChromaDB, then Ollama, then the static frontend.

The IPCC AR6 report was processed through a custom BeautifulSoup parser grouping paragraphs under parent headings into Section objects, with a sliding-window chunking strategy to preserve cross-paragraph context. Footnote blocks and HTML artefacts were filtered before ingestion, without this, footnote paragraphs inherited incorrect section IDs and polluted ChromaDB with ghost citations mapping to nonexistent sections. 222 HTML paragraphs were embedded into 224 ChromaDB chunks.

Citations were a persistent challenge. The chatbot needed to extract IPCC section references (e.g. [2.1.1]) from generated answers and render them as interactive chips that navigated the embedded report viewer. This involved rewriting `citation_parser.py` to silently drop unmatched citations rather than rendering unhelpful error chips, and strengthening prompt instructions in `prompt_templates.py` with worked examples. The citation debugging thread ran across several days in early July.

4.3 Frontend Architecture and Features

The ClimateInsight frontend was designed with three main sections: a collapsible conversation sidebar, a central chat interface, and an IPCC report viewer. During development, session management issues caused duplicate conversations to appear in the sidebar, which were resolved by improving how conversations were stored and tracked. Building on the modular design approach established in Phase Two, the interface also included an encyclopedia panel for climate term definitions and supported multi-turn conversations by preserving chat history throughout each session.

4.4 Architectural Comparison with the Reference Implementation

ClimateInsight is an evolving Retrieval-Augmented Generation (RAG) chatbot inspired by the [Climate Academy Chatbot](#) (CABot). It currently uses Flask, Ollama, ChromaDB, MiniLM embeddings, and a static HTML/CSS/JavaScript frontend to provide a complete end-to-end RAG pipeline. While the reference implementation includes additional production features such as Celery, Redis, and multilingual retrieval, ClimateInsight is designed to gradually incorporate these capabilities while building on its existing strengths, including conversation history, inline citations, modular architecture, and TOML-based configuration.

5. Reflections and Conclusion

This internship was a valuable learning experience in full-stack AI systems engineering. Over eleven weeks, the work progressed from resolving development environment issues to designing and building ClimateInsight, a complete Retrieval-Augmented Generation (RAG) chatbot. Along the way, challenges such as Python version incompatibilities, rebuilding the Help window, and redesigning the initial ClimateInsight architecture provided important lessons and led to stronger solutions.

Working on an active open-source project also developed practical skills beyond the classroom, including Git branch management, resolving merge conflicts, and writing clear documentation for future contributors.

The project combined software engineering with climate science communication, making accurate citations, source transparency, and accessibility key design priorities. Rather than simply generating answers, ClimateInsight was designed to provide reliable, traceable information from the IPCC AR6 Synthesis Report.

By the end of the internship, ClimateInsight had evolved into a functional end-to-end RAG chatbot with a well-documented architecture and a clear roadmap for future development, providing a strong foundation for the next stage of the project.

Week	Dates	Focus	Key Output
1	Apr 23–24	Environment setup, first run	Version-pinned setup documentation
2	Apr 27–29	Python version debugging	Root cause identified, fix documented
3	May 4–8	Branch management, smoke testing	Cloudflare tunnel reference
4	May 11–15	Architecture deep-dive, image bug	Image rendering fix · Web client docs
5	May 16–20	Frontend planning & early fixes	API URL fix
6	May 21–27	Help window, EMF conversion	Help module · PNG conversion utility
7	Jun 1–7	Major UI overhaul	Frontend v1.1.1 · Collapsible sidebar
8	Jun 8–11	Encyclopedia, mobile, scrolling	Encyclopedia modal · Scrollbar fix · Mobile Responsiveness
9	Jun 12–30	Codebase refactoring, blog	Refactoring Cloudflare CABot · Blog post
10	Jul 1–4	ClimateInsight build	Base version
11	Jul 5–Present	ClimateInsight polish & fixes	Citation system fix · Encyclopedia fix

[ClimateInsight](#) · [Frontend Version of Cloudflare Chatbot](#) · [Refactored Code](#) · [Daily Documentation](#)